

Dobby: Product-Level Architecture for Solidity AI Auditing, Development, and Knowledge-Augmented Security Analysis

BIFROST FOUNDATION

Dobby is a Solidity-specialized AI agent system for smart-contract auditing, development assistance, bytecode analysis, invariant generation, exploit validation, and knowledge-augmented security review. The product is built on an OpenCode-based runtime and extended with a Bifrost-specific profile, custom agent and skill policy, Model Context Protocol tooling, local retrieval systems, internal documentation retrieval, benchmark harnesses, and a server mode suitable for integration with operational tools such as Watchtower. This report presents Dobby as a product-level technical system rather than as a source-code inventory. It describes the implementation strategy, RAG composition, context-engineering method, audit workflow, API composability, and safety boundaries. Sensitive information such as credentials, deployment addresses, endpoint values, source tree topology, and environment-specific configuration is intentionally excluded.

Additional Key Words and Phrases: security engineering, context engineering, applied artificial intelligence

1 Introduction

Smart-contract engineering needs a different class of AI assistant from ordinary software development. Solidity programs are compact, stateful, and high impact. A small mistake in access control, reentrancy handling, price accounting, bridge validation, storage layout, signature verification, or upgrade initialization can create direct asset loss. A useful assistant must therefore combine language-model reasoning with contract structure, exploit-pattern memory, project context, validation tools, and a strict false-positive discipline.

Dobby addresses this requirement by turning a general agent runtime into a Solidity-first security product. It is not only a prompt pack. It combines an OpenCode-based execution engine, an Oh-My-OpenCode orchestration layer, a Dobby profile, specialized skills, MCP tools, vulnerability retrieval, internal repository retrieval, internal documentation retrieval, benchmark workflows, and a server-facing API. The result is a repeatable engineering system: Dobby can gather bounded context, analyze contracts, retrieve similar exploit patterns, generate invariants, call analysis tools, filter findings through an attacker model, and produce reports or patches that are suitable for human review.

The central design goal is excellence under operational constraints. Dobby must be better than a generic AI assistant because it understands Solidity-specific failure modes, Bifrost product conventions, cross-system dependencies, and internal operational knowledge. At the same time, it must not turn private repositories, runbooks, credentials, deployment values, or incident context into public output. This report therefore focuses on architecture and methods, not on confidential code layout or operational secrets.

2 Problem and Product Requirements

Dobby is motivated by a practical gap in smart-contract security workflows. General AI assistants can explain a function or draft a contract, but they typically lack four properties required for production security review. They do not reliably load the right project context. They often confuse privileged admin behavior with exploitable vulnerabilities. They can miss economic and cross-system invariants. They also tend to produce natural-language claims without reproducible evidence.

A production-grade Solidity assistant must satisfy a stricter bar. It must understand contract state, modifiers, inheritance, events, storage layout, external calls, bytecode selectors, control-flow structure, proxy upgrade rules, and DeFi-specific asset movement. It must retrieve similar exploit patterns and internal design context before reasoning. It must identify whether an issue is reachable by an unprivileged attacker, whether the attacker has a rational benefit, and whether the impact is meaningful. It must provide validation artifacts when a high-risk

claim is made. Finally, it must keep internal knowledge private while still using that knowledge to improve diagnosis.

Table 1. Product requirements that distinguish Dobby from generic AI coding assistance.

Requirement	Generic assistant pattern	Dobby product behavior
Domain context	Prompt-only Solidity awareness	Durable Solidity profile, security skills, MCP tools, and retrieval before reasoning
Audit method	Free-form review	Invariant-first workflow with detector, reasoner, verifier, and validation phases
Evidence	Natural-language assertion	AST, bytecode, knowledge graph, retrieval hits, generated tests, and proof sketches
False positives	Often unfiltered	Attacker-model, entry-point reachability, privilege, and motivation checks
Internal knowledge	Manual copy-paste	Governed repository and documentation RAG with bounded outputs
Operational use	Chat-only workflow	CLI, API server, browser chat, automation, report rendering, and benchmark harnesses

3 System Architecture

Dobby is organized as a layered agent system. The lower layer is an OpenCode-based runtime that provides session execution, model access, tool calls, file operations, and server mode. The orchestration layer extends that runtime with Oh-My-Opencode concepts such as the Sisyphus orchestrator, specialized agents, delegation rules, and role-specific model selection. The Dobby profile sits above that foundation and injects Solidity-specific prompts, RAG-first rules, skills, tool permissions, MCP configuration, and safety constraints.

The tool layer is exposed through a Solidity-focused MCP server. It provides compiler structure, bytecode analysis, selector decoding, control-flow and knowledge-graph construction, NatSpec and lint checks, storage-layout analysis, symbolic execution helpers, invariant generation, exploit proof generation, vulnerability retrieval, repository search, and internal documentation search. These tools make Dobby materially different from a raw chatbot: the model is surrounded by machine-readable evidence and repeatable workflows.

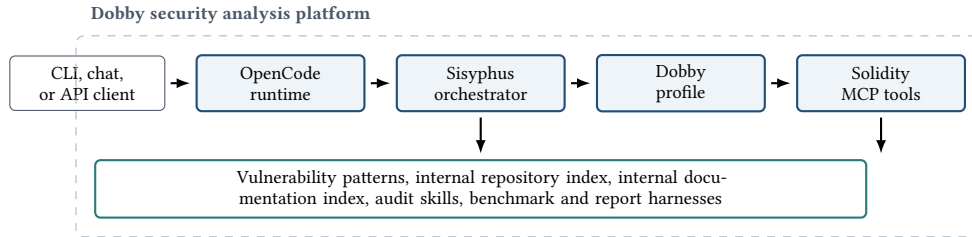


Fig. 1. High-level Dobby architecture. The OpenCode runtime provides the execution substrate; the Dobby profile, skills, MCP tools, and retrieval systems specialize it for Solidity security work.

This layering makes Dobby maintainable. OpenCode remains the execution substrate. Oh-My-Opencode supplies orchestration and delegation. Dobby-specific behavior lives in profile, skills, prompts, MCP tools, and retrieval packages. That separation allows upstream runtime improvements to be absorbed while the Solidity policy layer evolves independently.

4 Implementation Strategy

Dobby is implemented by wrapping and extending the OpenCode ecosystem rather than by building a custom agent runtime from scratch. The implementation keeps the agent engine, orchestration plugin, Dobby profile, MCP server, retrieval packages, CLI, chat surface, and server launcher as separate product responsibilities. The runtime can be installed for local terminal use, started as an API server, or accessed through a browser chat interface. In each mode, the same profile and MCP tool surface are loaded so the security behavior is consistent.

The OpenCode foundation gives Dobby several properties that would be costly to rebuild independently: session management, model-provider abstraction, tool execution, streaming events, file operations, and a server mode. Dobby extends this foundation by injecting a Solidity profile at startup. The profile selects the default orchestrator, configures a fast API-oriented agent for time-boxed server responses, enables the Solidity MCP server, and registers the Dobby prompt and skills. Runtime configuration is patched at launch so a developer or server environment can load the same profile without committing machine-specific paths.

The agent layer follows a deliberate division of labor. Sisyphus acts as the orchestrator: it classifies the request, decides whether the task is audit, development, deployment, testing, bytecode analysis, internal-contract search, or documentation search, and routes to the right workflow. Specialist roles then enforce different constraints. Development tasks may edit code and run tests. Audit tasks are treated as evidence-gathering and validation work. Repository and documentation search tasks are routed through retrieval tools instead of manual prompt stuffing. A separate high-reasoning validation role is used for severe findings so critical claims are checked before escalation.

Table 2. Implementation responsibilities in the Dobby product stack.

Layer	Responsibility
OpenCode runtime	Session execution, server mode, model-provider abstraction, tool calls, streaming events, and local file operations.
Orchestration plugin	Sisyphus-style request classification, agent delegation, role separation, and model selection.
Dobby profile	Solidity persona, RAG-first rules, safety constraints, MCP configuration, skill routing, and product-specific prompts.
MCP tool server	Compiler, bytecode, storage, quality, invariant, exploit, retrieval, and documentation tools.
Retrieval packages	Precomputed vulnerability, repository, and documentation indexes with local embedding support.
Harnesses	Setup, API-server launch, benchmark evaluation, report rendering, hot reload, and debugging workflows.

5 RAG Composition

Dobby uses retrieval-augmented generation as a product architecture, not as a single search box. The system has three retrieval memories. The first is a vulnerability memory containing curated smart-contract exploit patterns and audit heuristics. The second is an internal repository memory that indexes contract snippets and product code context. The third is an internal documentation memory that indexes architecture notes, API specifications, runbooks, procedures, and product design material. These memories are exposed to the agent through MCP tools so retrieval is explicit, bounded, and logged as part of the reasoning workflow.

The vulnerability memory helps Dobby answer the question “what does this contract resemble?” It is organized around exploit classes, service types, and expected invariants. During audit work, Dobby can retrieve similar historical failure patterns before producing findings. This improves recall because the agent is not relying only on model memory, and it improves precision because the retrieved pattern gives the agent a concrete invariant to test.

The repository memory helps Dobby answer the question “how does this company usually implement this pattern?” Instead of asking users to paste private repositories into a prompt, Dobby performs semantic search over indexed internal code and returns bounded snippets. The documentation memory answers a different question: “what was the intended design or operating procedure?” It indexes curated internal documents, classifies them by document type, chunks them by section, embeds them with local-first models, and applies metadata filters such as document type, space, label, and recency. This allows Dobby to combine code evidence with architecture intent when diagnosing a bug or answering an operational question.

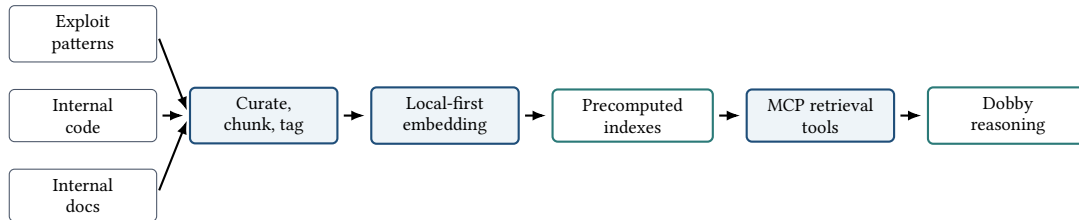


Fig. 2. Dobby’s RAG composition. Vulnerability, repository, and documentation memories are curated and embedded into bounded indexes, then exposed through MCP tools rather than pasted directly into prompts.

The important engineering choice is local-first retrieval. Internal code and internal documents can be embedded with local models so proprietary material does not need to leave the operating environment. The runtime can still support external embedding providers when explicitly configured, but the default architecture is designed around privacy-preserving local indexes. This makes RAG useful for sensitive security work while keeping public reports bounded to the evidence required for a claim.

6 Context Engineering

Dobby’s main advantage is context engineering: the system decides what the model should know before it answers. Context is split into durable policy, retrieved memory, task-local evidence, and output constraints. Durable policy includes the Solidity security persona, coding guidelines, audit workflow, attacker model, false-positive filters, tool-use expectations, and Bifrost engineering conventions. Retrieved memory comes from vulnerability, repository, and documentation RAG. Task-local evidence includes the contract under review, compiler output, traces, bytecode analysis, or user-provided error context. Output constraints prevent sensitive internal knowledge from leaking into reports.

The context strategy is trigger-driven. When a user mentions an internal protocol, product, repository family, operational issue, or company-specific term, Dobby is instructed to search internal code first and to search internal documentation when architecture, API, runbook, or incident context is relevant. For complex product questions, it combines both searches: code retrieval answers what the system does, while documentation retrieval answers why it was designed that way. This combined search strategy is especially important for diagnosis, where symptoms often arise from a mismatch between code behavior, operational assumptions, and service runbooks.

The same context engineering is applied to development and audit tasks. Development prompts load implementation constraints such as Checks-Effects- Interactions, explicit validation, event emission, NatSpec, gas-aware patterns, and mandatory test or compile loops. Audit prompts load service-type invariants, historical exploit patterns, entry-point reachability checks, privilege assumptions, and high-severity validation. Bytecode

tasks load selector, CFG, and decompilation context. Deployment and storage tasks load upgrade and storage-layout constraints. The product therefore changes the information environment around the model based on task type rather than using one universal prompt for every problem.

7 Agent and Skill Design

Dobby’s custom agent and skill design captures operating know-how that would otherwise live only in senior engineers’ heads. The design principle is to make each workflow explicit: when the user asks for an audit, Dobby should not casually mix implementation, speculation, and reporting. It should follow an audit workflow. When the user asks for development, Dobby should preserve project patterns, write tests, and run verification. When the user asks about an internal protocol, Dobby should retrieve internal code and docs before answering. These are product constraints, not stylistic preferences.

The skills encode concrete rules. Audit skills require an attacker model with no assumed key compromise, no assumed malicious privileged actor, a reachable entry point, a rational attacker benefit, and meaningful victim impact. Cross-system audit skills focus on state mapping mismatches, multi-actor race conditions, external call semantic gaps, and state divergence between EVM and non-EVM systems. Development skills require CEI ordering, explicit validation, event emission, NatSpec, gas-aware implementation, and compile/test loops. Testing skills generate unit, fuzz, invariant, integration, and fork tests depending on risk. Quality skills check lint, NatSpec, storage layout, and upgrade safety. Bytecode skills route source-incomplete analysis through selector decoding, CFG construction, and decompilation.

Table 3. Examples of Dobby-specific agent and skill constraints.

Area	Encoded know-how
Audit validation	Reject findings that depend on key compromise, malicious admins, unreachable entry points, self-harm, or economically irrational grieving.
Internal context	Use repository RAG for implementation evidence and documentation RAG for architecture, API, runbook, and incident context.
Development	Follow CEI, preserve existing patterns, document interfaces, emit meaningful events, generate tests, and verify with the project toolchain.
Cross-system review	Compare contract state assumptions with external systems, bridges, relayers, or operational documents.
Severe findings	Escalate critical and high claims through stricter validation and, when possible, reproducible tests or proof sketches.

This skill layer is a major source of product quality. It changes Dobby from a general assistant that happens to know Solidity into an opinionated security tool that executes repeatable workflows. The system can still use a powerful language model, but the model is constrained by task routing, retrieval, evidence requirements, validation gates, and report discipline.

8 Audit and Harness Engineering

Dobby’s audit methodology is invariant-first. The workflow begins by classifying the service type and identifying the properties that should hold: for example, lending solvency, vault share accounting, bridge state consistency, staking reward conservation, access-control separation, or upgrade storage preservation. Historical exploit retrieval supplies known patterns, while project-specific repository and documentation retrieval supply business logic and integration assumptions. The agent then looks for invariant violations rather than producing an unstructured list of possible issues.

The audit workflow is implemented as a harness around model reasoning. It performs context gathering, structural analysis, retrieval, candidate detection, reasoning, verification, high-risk validation, and report generation. This is important because language models are probabilistic. Dobby does not treat a plausible explanation as a finding. A reported issue should survive an attacker-model check, entry-point reachability analysis, privilege review, economic motivation review, and evidence check. High-severity findings can be converted into tests, exploit proof sketches, or deeper symbolic execution targets when appropriate.

The same harness mindset applies to development assistance. A patch is not finished because it looks reasonable. Dobby’s development workflows encourage compile, test, gas, trace, and quality checks. The report and benchmark harnesses make the system measurable: Dobby can be evaluated against known vulnerable examples, clean examples, and product-specific regression cases. This allows the team to improve prompts, retrieval, tools, and validation filters based on observed false positives and missed findings.

9 Dobby API Server and Watchtower Composability

Dobby can run as an API server in addition to a terminal agent. Server mode exposes session creation, asynchronous prompt submission, session status, message retrieval, model and agent discovery, cancellation, and event streaming for real-time UI updates. The browser chat surface is one client of this API, but the same pattern can support other internal systems. This is the key integration point for Watchtower composability.

Watchtower observes operational state, alerts, dry-run results, verification failures, query outputs, and other system events. Dobby can complement that surface by acting as a diagnosis layer. When Watchtower reports an unexpected error, a failed verification, a suspicious state transition, or an incident symptom, an integration can open a Dobby session, pass the bounded alert context, and ask for a diagnosis. Dobby can then retrieve internal runbooks, protocol documentation, related contract patterns, and historical failure classes before producing a structured explanation. The expected result is not an automatic final judgment; it is a more accurate first diagnosis that uses internal knowledge unavailable to generic assistants.

The potential is strongest for cross-system failures. Many operational bugs are not isolated Solidity defects. They involve observer state, relay behavior, bridge status transitions, off-chain indexing, API expectations, internal runbooks, and product-specific invariants. Dobby’s API server gives Watchtower a composable AI layer that can combine the live symptom with internal context: what this error usually means, which subsystem owns it, which invariant may be broken, what evidence should be checked next, and what remediation path is recommended. This turns Dobby into a company-aware diagnostic assistant rather than a detached code reviewer.

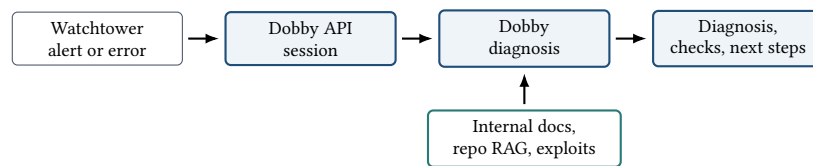


Fig. 3. Potential Watchtower composability. Watchtower supplies bounded operational context; Dobby enriches the diagnosis with internal knowledge and security reasoning.

10 Reliability, Safety, and Privacy

Dobby handles sensitive engineering context, so privacy and output discipline are part of the architecture. Credentials, authentication material, deployment addresses, private endpoint values, and repository topology must

remain local to the operating environment. Internal retrieval can inform analysis, but public reports should include only the evidence needed to support a technical claim. This is why Dobby's report generation is framed around product-level architecture, findings, invariants, and remediation rather than raw source inventory.

Operational safety also depends on least privilege and review. Tool access, repository retrieval, documentation retrieval, browser-enabled workflows, and server mode should be treated as governed surfaces. Generated artifacts should be reviewed before publication. Audit reports, patches, benchmark logs, and tool transcripts can contain sensitive context even when the model is behaving correctly. Dobby therefore improves internal reasoning while keeping external artifacts intentionally narrow.

Finding quality is another safety property. A Dobby finding should identify the vulnerable behavior, affected entry point, attacker capability, impact, evidence, and validation status. It should avoid claims that depend on compromised administrator keys, malicious privileged actors, unreachable internal functions, or self-harm-only behavior. This discipline matters because AI-generated security reports can otherwise create expensive review noise.

11 Conclusion

Dobby is a Solidity-specialized AI security and development platform built by combining an OpenCode-based runtime with Bifrost-specific context engineering, RAG, MCP tooling, agent policy, and workflow harnesses. Its strength is not one component in isolation. The product is excellent because the pieces reinforce each other: the runtime provides execution and API surfaces, the profile injects Solidity and Bifrost policy, the skills encode senior-engineering constraints, the retrieval systems provide exploit and internal context, and the harnesses turn model output into reviewable engineering artifacts.

This architecture lets Dobby move beyond ordinary prompt-only assistance. It can use company knowledge without exposing it, reason about Solidity and cross-system invariants, validate findings before escalation, assist development with project-aware constraints, and integrate with operational systems such as Watchtower through the API server. The result is a product that is valuable not merely because it calls a strong model, but because it constructs the right information environment around that model and applies the right engineering discipline to its output.